

UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

Resource Map

CSCE 629 Analysis of Algorithms — module-by-module mapping onto free university courseware

HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

Primary sources for the whole course

Resource	Role in this course
MIT 6.046J Design and Analysis of Algorithms (OCW, full video)	Primary lecture series. Graduate level, closely matched to this course's order. Watch the mapped lecture before each module.
MIT 6.006 Introduction to Algorithms, Spring 2020 (OCW, full video)	The undergraduate predecessor. Use it when a 6.046 lecture assumes background you do not have — particularly for hashing, graph search, and basic DP.
Jeff Erickson, Algorithms (free book)	The best free algorithms textbook in existence, and unusually good on <i>how to think</i> rather than only what is true. Primary written reference for this course.
Tim Roughgarden, Algorithms Illuminated (free videos)	A third voice, strong on intuition and on why each analysis is structured the way it is.
Competitive Programmer's Handbook (free)	Implementation-focused. Use for the practical details the theory courses skip, and pair it with the CSES problem set.
CSES Problem Set (free, auto-graded)	300 problems ordered by technique. The best free source of exercises with immediate feedback.

Module-by-module map

Module 01 — What Analysis Means

Resource	What it covers for this module
MIT 6.046J Lecture 01 — Course Overview, Interval Scheduling	Sets the level and the style of argument the course expects.
MIT 6.006 Lecture 01 — Algorithms and Computation	Gentler entry point covering models and asymptotic notation carefully. Start here if the notation is not yet automatic.
Jeff Erickson, Algorithms — Chapter 0 and the Introduction	Unusually honest about what analysis is for. Read the introduction even if you skip the chapter.
Latency Numbers Every Programmer Should Know	The constants the RAM model hides, in one page. Worth memorising the orders of magnitude.

Module 02 — Divide and Conquer, and Recurrences

Resource	What it covers for this module
MIT 6.046J — Divide and Conquer, Master Theorem	The master theorem derived from the recursion tree rather than stated, plus Strassen and closest-pair.
MIT 6.006 Lecture — Divide and Conquer	Gentler treatment with more worked recurrences.
Jeff Erickson, Algorithms — Chapter 1 (Recursion)	The best written treatment of recursion trees, with unusually good exercises.
Tim Roughgarden — Karatsuba and the Master Method	Part 1 covers both at an unhurried pace, with the intuition for why the three cases exist.

Module 03 — Randomization and Expectation

Resource	What it covers for this module
MIT 6.046J — Randomized Algorithms, Quicksort analysis	The indicator-variable analysis of quicksort done carefully, plus Karger's min-cut.
Jeff Erickson, Algorithms — Randomized Algorithms chapter	Excellent on the Las Vegas / Monte Carlo distinction and on treaps.
Tim Roughgarden — Randomized Selection and QuickSort	Part 1. Unhurried derivation of the $2/(j-i+1)$ probability.
Mitzenmacher & Upfal, lecture slides (free)	Concentration bounds with worked applications, if you want the probability theory tightened.

Module 04 — Sorting and Selection: Limits and Workarounds

Resource	What it covers for this module
MIT 6.006 — Sorting lower bound, Counting and Radix Sort	The decision tree argument presented carefully, with the linear-time sorts as the follow-on.

Resource	What it covers for this module
Jeff Erickson, Algorithms — Sorting chapter	Good on what the lower bound does and does not constrain.
Tim Peters — Timsort description (CPython source)	The design document for Timsort, written by its author. An unusually clear piece of engineering writing — read it.
Orson Peters — Pattern-defeating quicksort	How Rust's unstable sort resists adversarial patterns. Short and readable.

Module 05 — Hashing and Amortized Analysis

Resource	What it covers for this module
MIT 6.006 — Hashing with Chaining, Table Doubling, Amortization	Both halves of this module, carefully done, including the potential method.
MIT 6.046J — Amortized Analysis	The graduate treatment, with the potential method applied to more demanding structures.
Jeff Erickson, Algorithms — Hash Tables chapter	Universal hashing done properly, with the construction and the proof.
Google — Swiss Tables design notes (Abseil)	How a state-of-the-art open-addressed hash table is actually built, and why it is laid out the way it is. Short and worth reading.

Module 06 — Graph Search and Decomposition

Resource	What it covers for this module
MIT 6.006 — Graphs, BFS, DFS, Topological Sort	Careful development of both traversals and the properties each yields.
Jeff Erickson, Algorithms — Depth-First Search chapter	The best free treatment of edge classification and SCCs, with proofs that are actually readable.
Competitive Programmer's Handbook — Graph chapters	Implementation-focused, with the practical details (CSR layout, iterative DFS) that theory courses omit.
CSES Problem Set — Graph Algorithms section	Around 35 auto-graded problems covering exactly this module. The best free practice available.

Module 07 — Shortest Paths

Resource	What it covers for this module
MIT 6.006 — Dijkstra, Bellman-Ford, DAG shortest paths	Three lectures covering this module, with the relaxation framing made explicit.
Jeff Erickson, Algorithms — Shortest Paths chapter	Presents all of them as one generic relaxation algorithm with different orderings. The best treatment of this framing anywhere.

Resource	What it covers for this module
Amit Patel — Introduction to A* (Red Blob Games)	Interactive, and the clearest explanation of admissibility and heuristic quality available. Directly useful for game work.
CSES Problem Set — Shortest Path section	Auto-graded problems covering every algorithm here.

Module 08 — Minimum Spanning Trees and Union-Find

Resource	What it covers for this module
MIT 6.046J — Greedy Algorithms, Minimum Spanning Trees	The cut property proved carefully, with both algorithms derived from it.
Jeff Erickson, Algorithms — Minimum Spanning Trees chapter	Presents Kruskal, Prim, and Borůvka as one generic algorithm. Worth reading for that unification alone.
MIT 6.006 — Union-Find / Disjoint Sets	The data structure with the amortised analysis at an accessible level.
CSES Problem Set — Graph section (Road Reparation, Road Construction)	Direct practice on MST and union-find.

Module 09 — Greedy Algorithms and Exchange Arguments

Resource	What it covers for this module
MIT 6.046J — Greedy Algorithms	Interval scheduling and the exchange argument, done carefully.
Jeff Erickson, Algorithms — Greedy Algorithms chapter	Unusually good on how to <i>find</i> the right greedy rule and on the discipline of trying to break it first. Includes several instructive wrong rules.
Tim Roughgarden — Greedy Algorithms (Part 3)	Huffman coding and scheduling, with the proofs spelled out at length.
CSES Problem Set — Sorting and Searching section	Many of these are greedy problems in disguise. Good practice at recognition.

Module 10 — Dynamic Programming

Resource	What it covers for this module
MIT 6.006 — Dynamic Programming I–IV	Four lectures built around the 'five easy steps' framework, which is close to the four questions used here. The best structured introduction available.
Jeff Erickson, Algorithms — Dynamic Programming chapter	Outstanding on how to <i>find</i> the subproblem, with many worked examples and an honest account of the false starts.
Tim Roughgarden — Dynamic Programming (Part 3)	Knapsack, sequence alignment, and optimal BSTs at an unhurried pace.

Resource	What it covers for this module
CSES Problem Set — Dynamic Programming section	19 problems ordered by difficulty, auto-graded. The single best free source of DP practice.

Module 11 — Network Flow and Matching

Resource	What it covers for this module
MIT 6.046J — Network Flow, Max-Flow Min-Cut	Two lectures: the theorem with proof, then the reductions.
Jeff Erickson, Algorithms — Maximum Flow and Applications chapters	The best free treatment of the reductions, which is the part worth studying hardest.
Boykov & Kolmogorov — graph cuts in vision (free preprint)	How min cut became a standard tool in image segmentation. Directly relevant if you are heading toward CSCE 748.
CSES Problem Set — Graph section (flow problems)	Auto-graded practice on matching, disjoint paths, and min cut.

Module 12 — NP-Completeness and Reductions

Resource	What it covers for this module
MIT 6.046J — Complexity, Reductions, NP-completeness	Three lectures covering the definitions, the reduction technique, and several worked hardness proofs.
Jeff Erickson, Algorithms — NP-Hardness chapter	The clearest free treatment, with an unusually good discussion of how to pick which problem to reduce from and many worked gadgets.
Karp — Reducibility Among Combinatorial Problems (1972, free)	The original 21 reductions. Short, readable, and historically the paper that made the theory practical.
Complexity Zoo	A catalogue of complexity classes, for when you encounter one you do not recognise.

Module 13 — Approximation Algorithms

Resource	What it covers for this module
MIT 6.046J — Approximation Algorithms	Vertex cover, set cover, and TSP with the ratio proofs done carefully.
Williamson & Shmoys, The Design of Approximation Algorithms (free PDF)	The standard graduate text, released free by the authors. Organised by technique, which is the right way round.
Jeff Erickson, Algorithms — Approximation Algorithms chapter	A shorter introduction with the key proofs, good as a first pass.
Vazirani, Approximation Algorithms (author's page)	The other standard reference, strong on LP duality and primal-dual methods.